EPL Foul Win Probability Model

Author: EPL Analytics Team

Date: 2026-08-02

Table of Contents

Placeholder for table of contents 0

EPL Foul Win Probability Model

1 Executive Summary

1.1 Project overview

The EPL Foul Win Probability Model is a sports analytics solution developed to forecast whether a player who receives or recovers possession will later win a foul in the same possession. The solution uses event-level soccer data from StatsBomb and applies a Random Forest classification model within a reproducible Python pipeline.

1.2 Business problem

Football teams, scouts, and performance analysts need early indicators of fouls to inform tactical decisions, set-piece preparation, and player evaluation. Predicting foul-win probability helps identify players who are likely to draw fouls and assets that generate transition opportunities.

1.3 Solution

The solution ingests EPL event data, builds a binary classification target, engineers spatial and contextual features, trains a Random Forest classifier, and exposes results through a Streamlit dashboard. The model predicts a probability score that indicates the likelihood of a foul-win event.

1.4 Outcome

The pipeline delivers a complete analytics workflow from raw data to deployment-ready application. It produces evaluation metrics, model artifacts, and an interactive dashboard for stakeholder exploration.

1.5 Key achievements

- Constructed a robust target engineering methodology for foul-win prediction
- Designed domain-specific spatial and temporal features
- Trained and evaluated a Random Forest classifier with interpretable metrics
- Developed a user-facing Streamlit dashboard with automated scoring
- Integrated unit testing and CI guidance for a production-grade pipeline

2 Introduction

2.1 Background

Sports analytics has transformed football by enabling data-driven decisions across scouting, performance, and tactics. The ability to quantify player behavior during possession provides competitive advantage.

2.2 Sports Analytics

Modern sports analytics combines event tracking, machine learning, and domain expertise to generate actionable insights. In football, event data captures the sequence of actions that precede key outcomes.

2.3 Importance of foul prediction

A foul-winning event can halt an opponent's attack, enable set-piece opportunities, and reveal players who draw pressure effectively. Accurate prediction supports defensive planning, match preparation, and player valuations.

2.4 Why machine learning

Machine learning enables predictive capabilities beyond descriptive statistics. It can capture complex interactions between event location, timing, possession context, and the evolving state of a match.

2.5 Real-world applications

- Tactical planning for coaches
- Opponent risk assessment
- Player scouting reports
- Set-piece threat analysis
- In-game decision support

3 Problem Statement

3.1 Business objective

Predict whether an event in which a player receives or recovers the ball will result in a foul being won later in the same possession.

3.2 Machine Learning objective

Build a binary classifier to estimate the probability of a foul-win outcome and provide actionable model performance metrics.

3.3 Classification problem

This is a binary classification problem with the target values:

- `1`: Player wins a foul within the same possession
- `0`: Player does not win a foul

3.4 Target definition

The target is derived from event sequences and possession tracking. It requires identifying a ball reception or recovery event and verifying whether a foul event occurs subsequently during that possession.

3.5 Success criteria

- High precision for predicted foul-win events
- Balanced recall to avoid missing important foul opportunities
- Strong ROC-AUC and PR-AUC for imbalanced classification

- Clear feature importance and explainability

4 Dataset Description

4.1 StatsBomb Open Data

The dataset is sourced from StatsBomb Open Data, which provides granular event logs for English Premier League matches.

4.2 Dataset statistics

- 380 EPL matches
- 1.31M+ event records
- 381,267 candidate events
- Target class is imbalanced

4.3 Matches

The dataset contains a complete season of EPL matches, with match metadata and event sequences for each fixture.

4.4 Events

Each event record includes event type, timestamp, pitch coordinates, player identity, and match context.

4.5 Target dataset

Candidate events are generated from ball receipt and ball recovery events. The target dataset is a filtered subset of the event log that is suitable for foul-win modeling.

4.6 Class imbalance

The positive class (foul-win outcome) is a minority. Imbalanced classification is a central challenge and motivates careful metric selection.

4.7 Candidate event generation

Candidate events are extracted by selecting events that represent possession acquisition, such as receptions and recoveries. Those events are then connected to later foul outcomes within the same possession.

4.8 Dataset schema

Important columns include:

- `event_id`
- `match_id`
- `player_id`
- `event_type`
- `x`

- `y`
- `period`
- `minute`
- `second`
- `possession_id`
- `outcome`
- `foul_id`

4.9 Important columns

- `x`, `y`: pitch coordinates
- `event_type`: reception, recovery, foul, etc.
- `possession_id`: grouping events into a continuous team possession
- `minute`, `second`: temporal position within the match
- `outcome`: event success or failure

5 Exploratory Data Analysis

5.1 Dataset overview

EDA establishes the shape of the dataset, feature ranges, event frequency, and target balance. It confirms that candidate events are concentrated in midfield and attacking third.

5.2 Missing values

The dataset is examined for missing values in coordinates, event types, and possession indicators. Missing data is handled through filtering and validation.

5.3 Event distributions

Event distribution analysis reveals the frequency of receptions, recoveries, passes, dribbles, and fouls. This helps validate the candidate event extraction pipeline.

5.4 Target distribution

The target distribution confirms class imbalance and guides model evaluation toward precision, recall, and AUC metrics.

5.5 Possession analysis

Possession analysis demonstrates how the target is conditioned on team possession length and event sequence. The model focuses on events that initiate or continue a possession.

5.6 Coordinate distributions

Coordinate analysis reveals the typical zones where foul-winning events occur. These distributions support the use of spatial features such as distance to goal and pitch zone.

5.7 Graphs to include

- Event type frequency histogram
- Target class bar chart
- Possession length distribution
- Pitch coordinate density map
- Temporal distribution of candidate events

6 Data Cleaning

6.1 Missing values

Events with missing critical fields such as coordinates, event type, or possession ID are removed from the analysis.

6.2 Validation

Data validation checks ensure possession IDs are consistent, event ordering is chronological, and event types are properly labeled.

6.3 Filtering

Candidate event filters remove irrelevant actions, such as set-piece restarts and non-possessory events.

6.4 Data quality checks

- Consistency of pitch coordinates
- Valid possession lifecycle
- No future-event leakage
- Correct event sequencing

7 Target Engineering

7.1 Explain in detail

Target engineering is the process of converting raw event sequences into a binary label that indicates whether a foul was won during the same possession after a ball acquisition event.

7.2 Ball Receipt

Ball receipt events are identified when a player receives a pass or gains possession directly from a teammate. These events mark a new or continuing possession.

7.3 Ball Recovery

Ball recovery events occur when a player regains possession by intercepting, winning a loose ball, or recovering from an opponent's action.

7.4 Possession tracking

Possession tracking groups events by `possession_id`, ensuring that subsequent foul events are attributed to the same team possession.

7.5 Finding later foul events

The algorithm scans the event sequence for any foul event that occurs after the candidate event within the same possession. If a foul is found, the event is labeled positive.

7.6 Binary target creation

The target label is set to `1` for candidate events followed by a foul in the same possession, and `0` otherwise.

7.7 Algorithm explanation

1. Identify candidate acquisition events. 2. Group events by possession. 3. For each candidate event, scan later events in the same possession. 4. If a foul event appears, assign label `1`. 5. Otherwise, assign label `0`.

7.8 Complexity

The target creation algorithm operates with linear complexity relative to event count once events are grouped by possession. Efficient grouping is essential to avoid quadratic scanning.

7.9 Pseudo-code

```
for possession_id, events in possession_groups.items():
    foul_found = False
    for event in reversed(events):
        if event.type == 'foul':
            foul_found = True
        if event.type in candidate_acquisition_types:
            event.target = 1 if foul_found else 0
```

8 Feature Engineering

8.1 Explain every feature

The feature engineering stage converts raw coordinates and event context into predictive inputs.

8.2 x

The `x` coordinate represents the distance from the defending goal line. It encodes pitch depth and attacking progression.

8.3 y

The `y` coordinate represents lateral position on the pitch. It encodes the width of the event and relative field positioning.

8.4 Distance to Goal

Distance to goal is a derived spatial feature that measures Euclidean distance from the event point to the goal mouth.

Formula: \[\text{distance n to n goal} = \sqrt{(120 - x)^2 + (40 - y)^2}\]

8.5 Goal Angle

Goal angle captures the available shooting/foul-winning angle from the event location.

Formula: \[\text{goal n angle} = \arctan\left(\frac{7.32/2}{120 - x}\right)\]

8.6 Pitch Zone

Pitch zone segments the field into defensive, midfield, and attacking thirds. It provides discrete contextual information.

Categories:

- Defensive third
- Middle third
- Attacking third

8.7 Match Time

Match time includes elapsed minutes and seconds. It helps capture temporal context, such as late-game foul patterns.

8.8 Event Encoding

Event encoding converts categorical event types into numeric signals for the model. Candidate events are distinguished by labels such as receipt and recovery.

8.9 Why each feature matters

- `x` and `y`: location contextualizes the risk/reward profile of possession events.
- Distance to goal: fouls are more valuable near the opponent goal.
- Goal angle: narrow angles reduce goal threat but can still generate fouls.
- Pitch zone: each third has different tactical foul dynamics.
- Match time: fouls may cluster in late-match scenarios.
- Event encoding: acquisition type influences foul likelihood.

8.10 Mathematical formulas

- Distance to goal: \(\sqrt{(120 - x)^2 + (40 - y)^2}\)
- Goal angle: \(\arctan\left(\frac{7.32}{2(120 - x)}\right)\)
- Pitch zone boundaries: `x < 40`, `40 <= x <= 80`, `x > 80`

8.11 Football intuition

Players who receive the ball in advanced positions and in the attacking third are more likely to draw fouls. Spatial context and possession state are essential to identify these opportunities.

9 Machine Learning Pipeline

9.1 Feature selection

The model uses engineered spatial and contextual features selected for predictive relevance.

9.2 Train/Test Split

A train/test split isolates validation data to ensure evaluation reflects generalization performance.

9.3 Random Forest

A Random Forest classifier is chosen for its robustness to feature scaling, ability to model non-linear relationships, and interpretability via feature importance.

9.4 Hyperparameters

The pipeline is designed to tune the following hyperparameters:

- number of trees
- maximum tree depth
- minimum samples per leaf
- maximum features per split

9.5 Training workflow

1. Load processed dataset 2. Engineer features 3. Split train and test sets 4. Train Random Forest 5. Evaluate on holdout data 6. Persist the trained model

9.6 Pipeline diagram

```
flowchart LR
    A[Raw Event Data] --> B[Candidate Event Extraction]
    B --> C[Target Engineering]
    C --> D[Feature Engineering]
    D --> E[Train/Test Split]
    E --> F[Random Forest Training]
    F --> G[Model Evaluation]
    G --> H[Streamlit Dashboard]
```

10 Model Evaluation

10.1 Confusion Matrix

The confusion matrix measures true positives, false positives, true negatives, and false negatives.

10.2 Accuracy

Accuracy reports the overall correctness of the model, though it is less informative under class imbalance.

10.3 Precision

Precision measures how many predicted foul-win events were actually correct.

10.4 Recall

Recall measures the model's ability to identify actual foul-win events.

10.5 F1

The F1 score balances precision and recall and is appropriate for imbalanced classification.

10.6 ROC-AUC

ROC-AUC evaluates the model's ranking ability over all classification thresholds.

10.7 PR-AUC

PR-AUC is more informative for the minority class because it focuses on positive prediction performance.

10.8 Classification Report

The classification report summarizes precision, recall, F1 score, and support for each class.

10.9 Feature Importance

Feature importance highlights the most influential predictors for foul-win likelihood.

10.10 Strengths

- Robust to noisy feature scales
- Interpretable feature ranking
- Stable performance for structured event data

10.11 Weaknesses

- May require tuning for class imbalance
- Limited capture of sequential temporal dynamics
- Random Forest probability calibration may need adjustment

10.12 Interpretation

Higher importance for spatial features suggests location is a key driver of foul-win events. Temporal and possession context provide additional predictive value.

11 Streamlit Application

11.1 Explain application architecture

The Streamlit app is a front-end interface that loads the trained model and feature engineering logic, accepts user inputs, and presents probability predictions and evaluation insights.

11.2 Prediction workflow

1. User enters event details 2. Feature engineering transforms inputs 3. Model predicts foul-win probability 4. App displays score and classification

11.3 Input controls

The dashboard exposes controls for:

- Event type
- Match period and time
- Pitch coordinates
- Possession number

11.4 Automatic feature engineering

The Streamlit app computes derived features such as distance to goal, goal angle, and pitch zone automatically.

11.5 Probability prediction

The app presents a probability score and offers a binary interpretation tied to a scoring threshold.

11.6 Visualizations

The dashboard includes:

- Performance metrics
- Feature importance plots
- Confusion matrix
- ROC curve
- Precision-recall curve

11.7 Performance dashboard

The performance tab summarizes model quality and confidence across the evaluation dataset.

11.8 Feature importance

The app exposes the relative importance of each feature to support stakeholder trust.

11.9 Professional screenshots placeholders

- architecture.png
- pipeline.png
- feature_importance.png

- confusion_matrix.png
- roc_curve.png
- precision_recall_curve.png

12 Software Engineering

12.1 Project architecture

The repository is organized into modular source files, notebooks, data assets, and tests.

12.2 Folder structure

The architecture supports data processing, modeling, evaluation, deployment, and documentation.

12.3 Modular code

Key components are separated into single-responsibility modules for data loading, target creation, feature engineering, model training, and evaluation.

12.4 Logging

The pipeline includes logging support for traceability and debugging.

12.5 Configuration

The project uses a centralized configuration module to manage paths, parameters, and environment settings.

12.6 Reusable classes

Reusable utilities and feature engineering classes support consistent transformation logic.

12.7 Error handling

The pipeline validates inputs and handles missing or invalid event data gracefully.

13 Testing

13.1 PyTest

The repository uses PyTest for unit testing across the data pipeline and feature generation logic.

13.2 Unit Tests

Tests verify data loading, target creation, feature engineering, and model evaluation functions.

13.3 Coverage

Coverage reporting ensures test scope across the source modules.

13.4 Linting

Python style and formatting should follow best practices with tools such as Flake8 and Black.

13.5 GitHub Actions

CI pipelines run tests and validate the repository on push and pull request events.

14 Challenges

14.1 Class imbalance

The minority positive class requires careful metric selection and thresholding.

14.2 Feature engineering

Creating stable spatial features requires domain knowledge and coordinate validation.

14.3 Spatial calculations

Distance and angle formulas must align with pitch dimensions and coordinate conventions.

14.4 Model interpretation

Interpreting feature importance in a sports context demands clear communication to non-technical stakeholders.

14.5 Deployment

Deploying the Streamlit app requires reliable model loading, dependency management, and user input validation.

15 Future Improvements

15.1 Gradient Boosting

Evaluate Gradient Boosting classifiers for improved predictive performance.

15.2 XGBoost

Integrate XGBoost for faster training and advanced regularization.

15.3 LightGBM

Explore LightGBM for efficient large-scale event modeling.

15.4 SHAP

Use SHAP values for local and global interpretability.

15.5 MLflow

Track experiments and model versions with MLflow.

15.6 Docker

Containerize the application for reproducible deployment.

15.7 Cloud Deployment

Deploy to cloud platforms using Streamlit Cloud, Azure, or AWS.

15.8 REST API

Expose model scoring through a REST API for integration with analytics platforms.

15.9 Real-time predictions

Extend the solution to support live match event feeds.

16 Conclusion

16.1 Lessons learned

- Domain-specific feature engineering is essential for sports prediction.
- Event sequence modeling enables meaningful target creation.
- Streamlit provides a rapid interface for stakeholder validation.

16.2 Business value

The model offers predictive visibility into foul-winning events, supporting tactical decision-making, player scouting, and set-piece planning.

16.3 Sports analytics impact

This project demonstrates how event data can generate actionable insights in football and support modern analytics workflows.

16.4 Future scope

The pipeline is well-positioned for advanced model experimentation, expanded event coverage, and production deployment.

17 References

- StatsBomb Open Data
- Scikit-learn documentation
- Streamlit documentation
- Python language reference
- Random Forest literature

- Sports analytics research